



Vibe Coding

调研分析

汇报人: Codex

日期: 2026.05.24



目录

01

背景与定义

从人机协作视角界定 vibe coding

02

方法与流程

拆解从想法到可运行原型的路径

03

收益与风险

分析效率提升和质量控制边界

04

落地展望

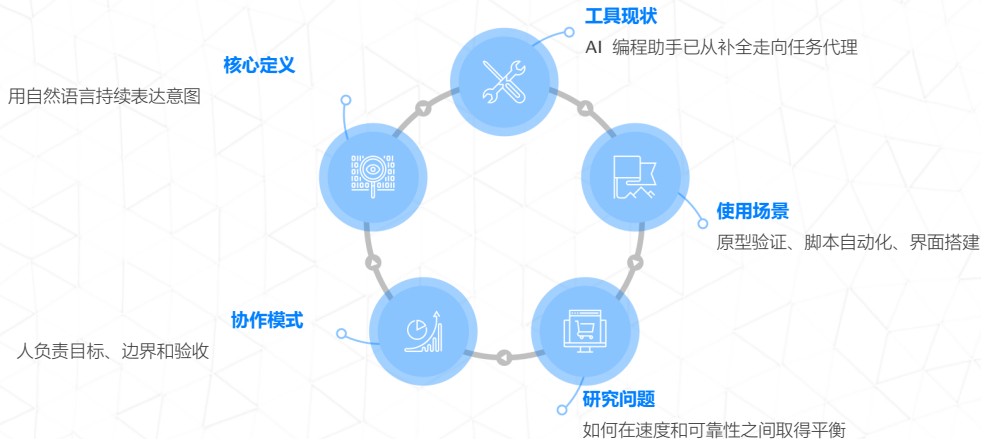
给出团队采用和治理建议

01

背景与定义



Vibe Coding 概述



为什么值得关注

需求侧变化

越来越多非专职开发者希望把想法快速变成可交互原型，传统开发流程显得偏重

供给侧变化

大模型可以理解上下文、生成代码并解释修改原因，使编程从写语法转向表达意图



02

方法与流程



Vibe Coding workflow



调研方法

本次调研以桌面研究和小样本实践为主，观察 AI 编程助手在需求澄清、代码生成、错误修复、样式还原和测试验证中的表现。重点不评价单一模型强弱，而是分析这种协作范式对个人和团队开发流程的影响。



关键词：
意图表达
快速原型
自动修复
人工验收

典型能力拆解



生成

根据目标描述生成页面、接口、脚本或测试样例，降低启动成本



修改

根据反馈进行局部重构和样式调整，适合连续迭代



验证

通过编译日志、截图和测试结果定位问题，减少盲目猜测



沉淀

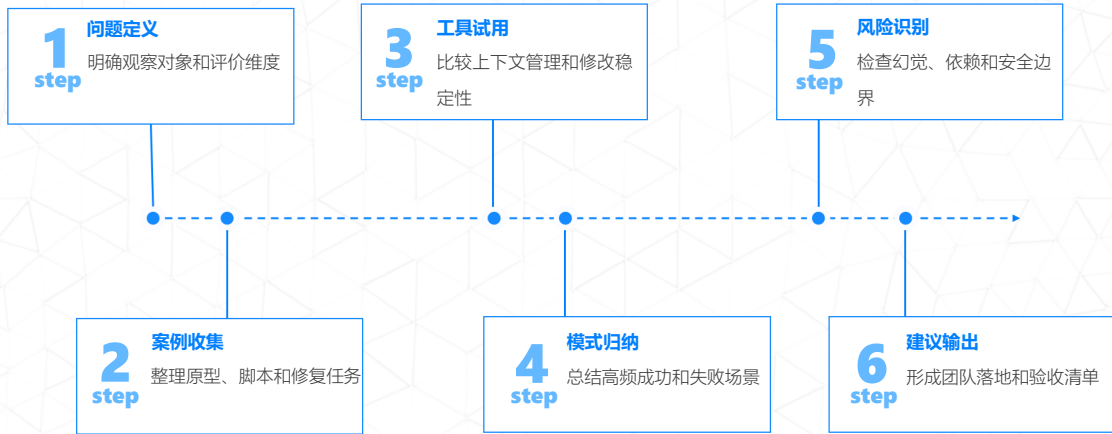
把有效提示、代码片段和流程规范沉淀为团队资产

03

收益与风险



调研过程



关键发现

Vibe coding 带来了什么

最大收益是把低确定性的探索阶段大幅压缩，让用户更快看到可运行结果。主要风险是模型会自信地生成错误实现，并在需求含混时扩大偏差。因此，vibe coding 更适合作为加速器，而不是替代工程判断。



04

落地展望





落地建议

团队不应只讨论是否使用 AI 编程，而应设计可验证的协作流程。建议从低风险任务开始，明确代码审查、测试运行、依赖记录和数据安全边界，让速度提升建立在可控质量之上。

实施路径

试点任务

选择脚本、页面和文档生成等低风险场景

知识沉淀

保存有效提示词和常见错误处理方式

验收标准

要求可运行、可复现、可解释和可回滚

治理机制

对敏感代码、许可协议和数据外发设红线

未来展望



人人原型

业务人员可以更快表达需求并验证想法



人机结对

开发者转向架构判断和质量把关



测试前置

自然语言验收标准会更早进入流程



工具融合

编辑器、终端和项目管理进一步打通



规范成熟

团队会形成 AI 协作的审查和合规标准



谢谢聆听

Vibe Coding Research

汇报人：Codex

日期：2026.05.24

